Consulting Case Study

Amrit Water WhatsApp AI Sales & Order Agent

Prepared by	Saloni Sisodiya, AI Solutions Consultant
Date	November 2026
Engagement type	Proof of concept for a packaged-water wholesale business in Ratlam, India
Important	Rates and orders are demo data. No production results are claimed.

1. Engagement summary

Item	Detail
Client profile	Packaged drinking water wholesaler in Ratlam. One product (500 ml), sold in cartons of 12
Problem	Orders and questions arrive on WhatsApp and are handled by hand: slow replies, missing order details, no reliable order record
Solution	A WhatsApp AI agent that quotes prices, takes and confirms orders, records them, and escalates sensitive cases to the owner
Stack	WhatsApp Cloud API, n8n, Google Gemini, n8n Data Tables
Approach	Version 1: fixed-flow assistant. Version 2: tool-using agent with memory, a price table and guardrails
Status	Working end to end on a test number. 14 scenarios tested, 7 defects found and fixed. Demo video, workflow export and documents are in the repository

2. The problem and the goals

The owner takes orders in WhatsApp chats. Each order needs five details (size, cartons, delivery location, customisation and phone number), and customers rarely send them all in one message. Quotes depend on quantity slabs and delivery rules, so answers vary with whoever replies. Orders are not recorded consistently.

Goal	Measure
Fast, consistent first response	Reply within seconds (2 to 3.5 seconds observed in testing)
Complete orders	All details captured before an order is saved; no duplicate rows
Accurate prices	Zero invented prices or totals in the test set
Owner stays in control	Alerts for every confirmed order and escalation; confirmed orders never cancelled by the bot
Orders recorded	Every order stored with a status

3. Discovery and requirements

The requirements were written as a Business Requirements Document (BRD v1 for the assistant, BRD v2 for the agent). The v2 requirements that mattered most:

- Collect order details one question at a time without repeating questions.
- Quote prices only from a rate table and compute every total with a calculator.
- Confirm an order only after an explicit customer yes.
- Never cancel a confirmed order automatically; send the request to the owner.
- Escalate complaints, discount requests and out-of-area delivery to the owner.
- Reply in English or Hinglish, matching the customer.

Business rules agreed

- 1 carton = 12 bottles; only 500 ml; bottles convert to cartons only when divisible by 12.
- Delivery within Ratlam only: ₹50 under 10 cartons, free from 10.
- Demo rate card: 1+ cartons ₹108, 10+ ₹102, 50+ ₹96 per carton, before GST.

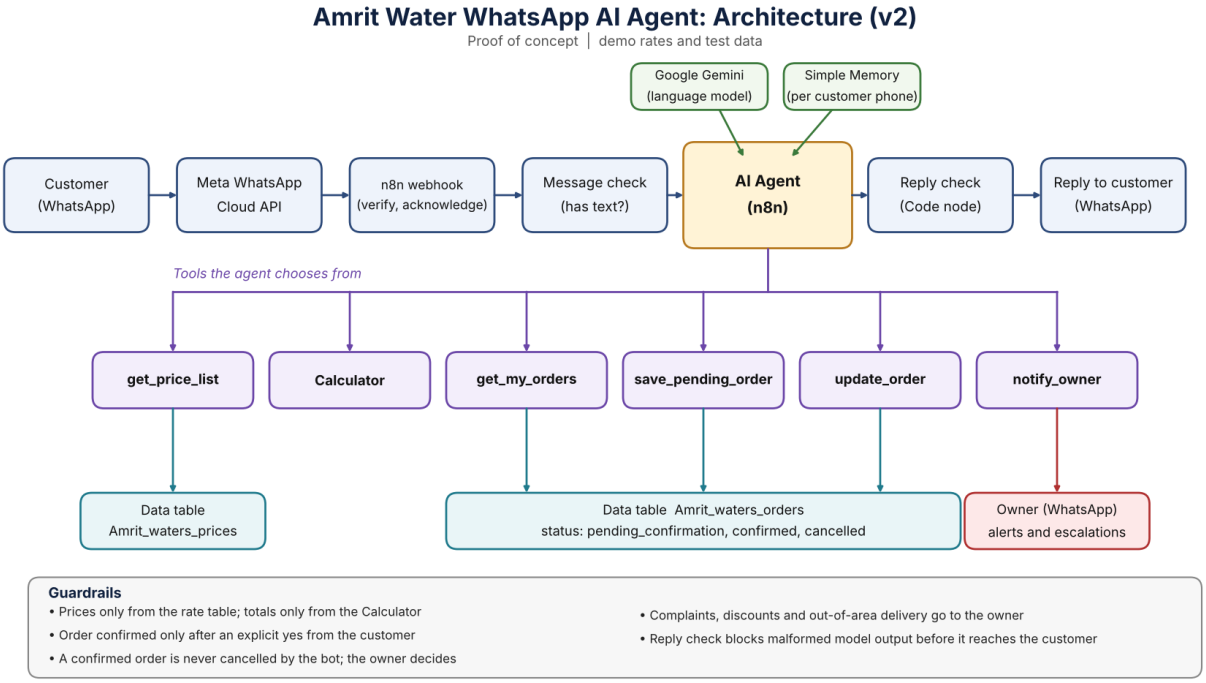
Assumptions

- Rates are demo values based on typical small-manufacturer listings, not Ratlam quotes.
- The owner and customers use different WhatsApp numbers.
- The proof of concept runs on free or trial tiers.

4. Options considered

Option	Strengths	Weaknesses	Verdict
A. Keyword chatbot	Cheap, predictable	Breaks on free text, Hinglish and corrections	Rejected
B. Fixed-flow LLM assistant (v1)	Simple, low cost	No memory; general questions unanswered; duplicate pending rows	Built first, then replaced
C. Tool-using AI agent (v2)	Handles edits, repeat orders, status, escalation across several turns	Needs guardrails and thorough testing	Selected
D. Managed WhatsApp commerce platform	Fast to launch	Monthly fee; limited custom pricing rules	A fair alternative if the client prefers a managed product

5. Solution design



The agent runs inside an n8n workflow. For each message it loads that customer's memory, decides which tool to call, and replies. A reply check blocks malformed model output before it reaches the customer.

Tool	Purpose
get_price_list	Reads the rate card from a data table
Calculator	Computes every total so the model never does its own arithmetic
get_my_orders	Looks up this customer's orders and statuses
save_pending_order	Creates the draft once all details are known
update_order	Edits a draft, confirms it after a yes, or cancels a draft
notify_owner	Sends the owner a WhatsApp alert

Key design decisions

- One agent with tools rather than multi-agent: easier to debug, explain and hand over.
- Rates in a table, not in the prompt, so the owner can change a price without touching the agent.
- A human stays in the loop for money, complaints and cancellations.
- Memory is per customer phone number; production should use database-backed memory.

6. Delivery approach

Phase	Activities	Deliverable	Typical effort for a real client
1. Discovery	Interview the owner; collect sample chats; agree rules and success criteria	Requirements and rules	0.5 to 1 day
2. Design	Options, architecture, data model, prompt draft	Solution design	0.5 day
3. Build	WhatsApp setup, workflow, tools, prompt	Working agent on a test number	2 to 3 days

Phase	Activities	Deliverable	Typical effort for a real client
4. Test	Scenario tests, defect fixes, owner review	Test report	1 day
5. Go-live	Production number, token, monitoring, owner training	Live agent and runbook	0.5 day
6. Hypercare	Review real chats and tune the prompt	Weekly summary	2 weeks

For this proof of concept, version 1 was built first, tested, and then upgraded to version 2 after its limits became clear (general questions unanswered, no memory, duplicate rows).

7. Testing and quality

Scenario-based testing through real WhatsApp conversations on a test number, with a second phone acting as the owner. A scenario passes only if the reply, the order row and the owner alert are all correct. All 14 scenarios passed: greeting, step-by-step order, quantity change, confirmation, payment choice, bottles to cartons, non-multiple of 12, large-quantity pricing, out-of-area delivery, complaint, cancelling a draft, cancelling a confirmed order, order status, and English/Hinglish switching.

#	Defect found	Root cause	Fix
D1	Agent failed on real WhatsApp messages	Prompt mapped to the mock-data shape	Remapped to the live message field; retested
D2	Wrong total (₹2,160 instead of ₹2,040)	Model did its own arithmetic	Totals only from the Calculator tool
D3	Rate-limit errors mid-order	Free-tier token limit	Paced tests, shorter memory, moved to Gemini; fallback model advised
D4	Wrong language and stale context	Memory mirrored earlier replies	Language rule on the latest message; fresh test sessions
D5	Bot claimed orders were ready for delivery	No delivery data exists	Report only fields in the table
D6	Owner alert reached the customer chat	Change not published; one number for both roles	Published; separate owner number
D7	A reply showed the model's internal reasoning	Reasoning-style model leaked its thinking	Output rule, reply-check step, model change

8. Risks and mitigations

Risk	Mitigation
Model invents prices or delivery claims	Rate table, Calculator tool, explicit rules, reply check, testing
Rate limits or outage	Fallback model and paid tier
Memory lost on restart	Database-backed memory
Owner alert blocked by WhatsApp's 24-hour rule	Approved message templates
Token expiry	Permanent system-user token and a rotation date
Customer data exposure	Store minimal fields; no secrets in the repository or screenshots
Change not published	Pre-release checklist in the runbook

9. Deployment and operations

Go-live checklist: production number registered, permanent token created, webhook and verify token set in Meta, workflow published, owner number tested, scenarios rerun on live, and a named monitoring owner. Monitoring means reviewing failed executions and spot-checking chats against the order table. Cost drivers are the n8n plan and execution volume (each message also triggers Meta status updates), model usage, and WhatsApp conversation charges; current pricing should be checked before quoting a client.

10. Handover

The client receives the sanitized workflow export, the table schemas, the system prompt, the BRD, solution design and test report, a runbook (change a rate, read alerts, pause and resume, rotate the token, publish checklist, troubleshooting), and a short walkthrough for the owner.

11. Value and ROI

No business results are claimed from the proof of concept. To estimate value for a real client, measure during discovery and compare after a pilot:

> Monthly saving = orders per month x minutes of staff time saved per order x staff cost per minute, plus avoided losses from missed or mistaken orders.

- Orders per month and average chat length
- Share of orders that arrive with missing details
- Reply delay at peak times
- Errors in quotes or records

12. Lessons learned

- Test with real message payloads early; mock data hid a mapping error.
- Never let the model do arithmetic that affects money; use a calculator tool.
- Keep a human in the loop for complaints, discounts and cancellations.
- Free tiers have rate limits that show up mid-conversation; plan a fallback.
- Conversation memory can carry old behaviour forward; test from clean sessions.
- Add an output check, because a model can occasionally leak its own reasoning.
- Workflow changes only take effect when published; make a checklist part of operations.

13. Next steps

- Run a supervised pilot with a real small business, with success criteria agreed before it starts.
- Replay 15 to 20 real past chats and compare the agent's answers with the owner's.
- Add a fallback model, database-backed memory, approved alert templates and automated regression tests.
- Later: payment links, stock checks, an order dashboard and voice-note support.